Agent Postmortem Generator

Part 3: Normalization Engine

One-line purpose: Convert validated generic trace input into the canonical, loss-aware trace package that downstream analysis can trust.

---

1. Title Page

Project Name: Agent Postmortem Generator
Part Number: 3
Module Name: Normalization Engine
Purpose: Turn validated input records into canonical, deterministic, provenance-preserving trace entities for downstream analysis.

2. Executive Overview

The Normalization Engine is the boundary between “validated but still source-shaped” trace data and the project-owned canonical trace model. Its job is to transform a generic JSON execution trace into a stable, framework-agnostic structure that the Analysis Engine can consume without knowing anything about the original agent framework, storage format, or logging quirks.

This module exists because validated input is still not analytically usable. Raw traces typically contain a mixture of nested records, inconsistent identifiers, partial timestamps, repeated state snapshots, and framework-specific event labels. None of that should leak into downstream code. The normalizer removes that variation by applying explicit, deterministic mapping rules and preserving provenance for every important derived field.

In the system pipeline, the Normalization Engine sits directly after Parser + Validation and directly before the Canonical Trace Model / Analysis Engine boundary.

Input -> Parser -> Validation -> Normalization Engine -> Canonical Trace Model -> Analysis -> Postmortem

The module has one purpose: produce a canonical trace package that is consistent, explainable, and safe for deterministic analysis. It does **not** interpret intent, infer blame, or reconstruct causality. It only prepares the evidence in a form that later modules can reason about.

3. Problem Statement

The project cannot perform trustworthy postmortem analysis if downstream modules must understand every source format’s quirks. That would force the Analysis Engine to carry parser knowledge, framework-specific branching, and bespoke cleanup logic. The result would be brittle, hard to test, and impossible to keep deterministic.

The exact problem this module solves is:

- source traces are structurally valid but not analytically uniform;
- equivalent concepts appear under different field names or nesting patterns;
- event order is often implicit rather than explicit;
- identifiers may be local, composite, duplicated, or partially missing;
- parent-child relationships may need normalization, but not interpretation;
- evidence must be preserved even when a field is unknown or incomplete.

This is hard because normalization sits in the awkward middle: it must be strict enough to protect the canonical model, but tolerant enough to avoid throwing away evidence that later helps explain a failure. The engine must distinguish between “recoverable irregularity” and “unsupported or invalid trace semantics” without guessing.

The MVP needs this module because the core product value depends on a stable canonical representation. Without normalization, every downstream module would have to implement its

own cleanup logic. That would destroy modularity, increase regression risk, and make the system impossible for 2 to 3 engineers to maintain.

4. Goals

Success for this module means that any validated generic JSON trace can be converted into the same canonical shape every time, regardless of record ordering or source noise.

Primary goals

Goal	What it means in practice
Deterministic output	The same input always produces the same canonical package, diagnostics, and ordering.
Canonical compatibility	Output matches the project-owned canonical trace model exactly.
Provenance preservation	Important normalized fields retain links back to their source path, source record, or raw value.
Conservative inference	The engine normalizes only what is explicit or mechanically derivable.
Loss visibility	If normalization drops, merges, or classifies anything, the result is reported in diagnostics.
Downstream cleanliness	Analysis code does not need to know about raw source shapes, framework labels, or parser internals.

What “done” looks like

- Every normalized entity has a stable canonical ID.
- Every meaningful field can be traced back to its source record or source path.
- Unknown or unsupported records are either retained as opaque evidence or rejected with a deterministic diagnostic.
- Relationship graphs are normalized into a single canonical direction and ordering.
- Output is versioned so future schema changes are explicit, not accidental.

5. Non-Goals

This module must stay narrow. Anything outside deterministic transformation belongs elsewhere.

Explicitly out of scope

- Parsing raw JSON text.
- Schema discovery or automatic schema generation.

- Framework-specific adapters.
- Causality reconstruction.
- Root cause analysis.
- Incident narrative generation.
- Visual rendering or report formatting.
- Storage, indexing, search, or remote synchronization.
- Any probabilistic or LLM-based inference.
- Cross-run comparison or drift detection.
- Telemetry ingestion beyond the MVP input contract.

Deferred to later versions

Deferred feature	Why it is deferred
OpenTelemetry normalization	It introduces a second source family and deserves its own adapter path.
Streaming normalization	Not needed for the MVP trace size and would complicate the state model.
Multi-run alignment	Requires a richer canonical identity model than the MVP needs.
Semantic enrichment	Anything that tries to infer missing meaning is analysis, not normalization.
PDF export	Output-layer responsibility, not normalization responsibility.

6. Responsibilities

This module owns the transformation from validated input records to canonical trace entities.

What this module owns

- Record classification into canonical entity types.
- Stable canonical ID generation.
- Reference resolution for explicit parent-child and related-record links.
- Deterministic ordering of records and events.
- Timestamp normalization where the source provides enough information.
- Preservation of raw source values, source paths, and record fingerprints.
- Construction of the canonical execution graph.
- Detection and reporting of duplicates, cycles, ambiguity, and unsupported record kinds.
- Emission of structured normalization diagnostics.

What this module must never own

- Parsing or repairing malformed JSON.
- Interpreting user intent or agent reasoning.
- Guessing missing parents, tool outcomes, or causal links.

- Applying framework-specific business rules.
- Building any user-facing report content.
- Mutating downstream analysis expectations by “helpfully” filling in gaps.
- Hiding ambiguity behind silent defaults.

A good rule: if a transformation can no longer be explained as a deterministic mapping from input evidence, it does not belong here.

7. Inputs

The Normalization Engine receives a validated, structured trace object from the Parser + Validation layer. It should not receive raw JSON text and should not attempt its own schema discovery.

Expected inputs

Input category	Examples	Required?	Notes
Run metadata	run id, trace version, source name	Yes	Provides top-level identity and provenance.
Ordered records	steps, tool calls, messages, state snapshots, errors, artifacts	Yes	The normalization engine operates over records, not text blobs.
Explicit references	parent ids, message ids, tool ids, correlation ids	When present	Used for graph construction and linkage.
Timestamp fields	created_at, started_at, ended_at, observed_at	When present	Normalized using a fixed precedence order.
Raw payloads	framework-specific or generic payload blobs	Often	Preserved as evidence, even when opaque.
Source paths	JSON pointer, dotted path, record index	Yes, ideally	Used for provenance and diagnostics.

Validation rules inherited from the previous layer

The module assumes the input has already passed structural validation. In practice, that means:

- the root object exists and is parseable;
- required top-level keys are present;

- record collections are iterable and finite;
- field types are broadly correct;
- the input is local, immutable, and non-streaming for the MVP.

Additional normalization-time assumptions

- The source order of records is meaningful and must be preserved as a fallback order key.
- Identity fields may be incomplete, but they are not arbitrary.
- Explicit relationships take precedence over inferred adjacency.
- Unknown record kinds should be preserved if they are structurally safe to carry forward.

Malformed input handling

The engine should fail fast for violations that make deterministic normalization impossible, such as:

- missing top-level run identity with no deterministic fallback;
- cyclic references in fields that are required to form an acyclic execution graph;
- non-finite record collections;
- conflicting record identifiers that cannot be reconciled deterministically.

Recoverable irregularities should not abort the entire run. Instead, they should produce diagnostics and a conservative canonical result.

Case	Behavior
Missing optional timestamp	Keep null, preserve source order, emit warning if relevant.
Unknown record kind	Retain as opaque extension record with diagnostic.
Duplicate record with identical payload	Deduplicate deterministically and report count.
Conflicting duplicate record	Keep first-seen record, report conflict.
Missing explicit parent reference	Leave relationship null; do not infer it from proximity.
Cycle in execution graph	Reject the later edge in source order and report the cycle.

8. Outputs

The module emits a canonical trace package plus structured diagnostics. It should not emit human prose or visual output.

Exact outputs

Output artifact	Description	Consumers
Canonical trace package	The normalized, project-owned representation of the execution trace.	Analysis Engine
Normalization diagnostics	Structured warnings/errors about loss, ambiguity, and unsupported cases.	CLI, logs, tests
Normalization summary	Counts of records, canonical entities, duplicates, dropped edges, and opaque records.	CLI, telemetry-free local reporting
Provenance index	Mapping from canonical fields back to source record locations and raw values.	Debugging, tests, future tooling

Data artifacts

The canonical package should be serializable to a stable JSON representation. The serializer is not the responsibility of this module, but the normalized object model must be serialization-friendly.

The package should include:

- schema version for the canonical model;
- schema version for the normalization contract;
- normalized run metadata;
- canonical entity collections;
- canonical relationship graph;
- provenance references;
- diagnostics and counters.

Human-readable artifacts

This module should not generate markdown reports, PDF content, or narrative explanations. Human-readable summaries belong to the output layer or CLI wrapper.

9. Internal Design

The engine should be small, explicit, and easy to test. It does not need a plugin system for the MVP.

Main subcomponents

Subcomponent	Responsibility
Record classifier	Assigns each validated source record to a

Subcomponent	Responsibility
	canonical record kind.
Identifier resolver	Maps raw identifiers and aliases to stable canonical IDs.
Temporal normalizer	Produces a deterministic ordering and normalizes timestamps when possible.
Relationship builder	Connects entities using explicit source references only.
Canonical assembler	Materializes the final canonical trace package.
Diagnostic collector	Records warnings, errors, and normalization decisions.

Workflow

Validated Input
  |
  v
Record classification
  |
  v
Canonical ID assignment
  |
  v
Timestamp and ordering normalization
  |
  v
Reference resolution
  |
  v
Graph assembly
  |
  v
Invariant checks
  |
  v
Canonical package + diagnostics

Internal design decisions

1. **Normalization is evidence-preserving, not evidence-rewriting.**
   Raw values should remain available whenever normalization changes meaning or drops detail.

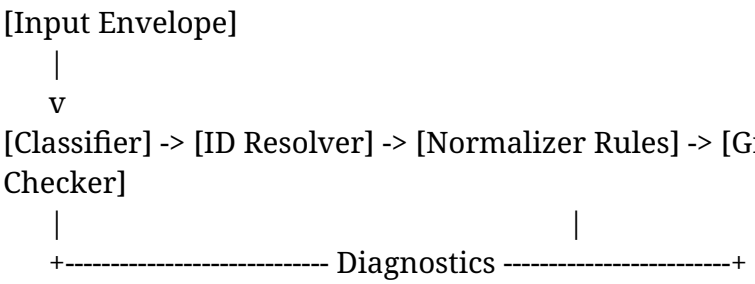
2. **Explicit relationships win.**
   The engine should not infer parentage from adjacency, nested structure, or labels unless the contract explicitly says so.

3. **Ordering must be deterministic.**
   If timestamps are absent or conflicting, use source order as the final tie-breaker. Never use randomness.

4. **Canonical IDs must be stable.**
   IDs should be derived from deterministic source identity plus role, not object memory or collection position alone.

5. **The output should be immutable.**
   Once the canonical package is produced, downstream code should treat it as read-only.

Layered internal flow



10. Data Model

The Normalization Engine does not redefine the canonical trace model, but it does own the mapping logic and the normalization metadata attached to it.

Core entities

Entity	Purpose	Key fields
NormalizationResult	Top-level output wrapper	schema version, canonical package, diagnostics, summary
CanonicalRun	Normalized execution container	run id, source ids, start/end bounds, source metadata
CanonicalRecord	Generic normalized event/step/message/tool record	canonical id, kind, timestamp, execution order, payload ref
CanonicalRelation	Explicit link between canonical records	relation type, source ref, from id, to id
CanonicalArtifactRef	Pointer to evidence retained from the source	raw path, raw value hash, mime/type hints
ProvenanceEntry	Field-level origin metadata	canonical field, source

Entity	Purpose	Key fields
		record, source path, raw value fingerprint
NormalizationDiagnostic	Warning or error emitted during transformation	severity, code, message, affected ids, source ref

Relationships

- One NormalizationResult contains exactly one normalized run package.
- One CanonicalRun contains many CanonicalRecord items.
- Each CanonicalRecord can participate in multiple CanonicalRelation edges.
- Each important canonical field should have an associated ProvenanceEntry.
- Diagnostics may reference one or more canonical IDs and one or more source records.

Why this model exists

The data model exists to make transformation explainable and testable. The downstream Analysis Engine should not need to wonder why a record exists, where a field came from, or whether a relationship was inferred. The provenance and diagnostic structures answer those questions directly.

11. Algorithms / Processing Logic

Normalization should be a rule-driven deterministic transform. It should not contain hidden heuristics.

Step-by-step processing

1. **Read the validated input envelope.**
   Confirm the root trace metadata exists and establish the normalization context.

2. **Index source records.**
   Build a deterministic iteration order and a lookup table for explicit identifiers and aliases.

3. **Classify each record.**
   Map the source record to a canonical kind using the fixed MVP record taxonomy.

4. **Assign canonical IDs.**
   Generate stable IDs using source identity, record role, and deterministic disambiguation rules.

5. **Normalize timestamps.**
   Apply a fixed precedence order for time fields. Preserve raw values when timestamps conflict or are missing.

6. **Resolve explicit references.**
   Convert parent-child, call-result, and correlation links into canonical relations when the source makes them explicit.

7. **Assemble the execution graph.**
   Materialize records and relationships into the canonical run graph.

8. **Check invariants.**
   Validate that required structural rules hold, such as acyclic execution edges where the canonical model requires them.

9. **Emit diagnostics and summary counts.**
   Record any lossiness, ambiguity, duplicates, unsupported records, or dropped edges.

10. **Return the canonical package.**
    The result should be ready for deterministic downstream analysis.

Decision rules

Situation	Rule
Multiple candidate timestamps	Use the highest-precedence explicit timestamp field, then source order as a tie-breaker.
Duplicate raw IDs	Keep first-seen by source order; mark later duplicates as conflicts.
Missing explicit parent	Leave link null; do not infer from nesting.
Unknown record kind	Preserve as opaque extension record if structurally safe.
Conflicting references	Reject the later conflicting reference and emit a diagnostic.
Cycle detected	Cut the later edge and record the cycle event.

Edge cases

- **Out-of-order records:** normalize using source order plus explicit timestamps, not arrival order.
- **Partial tool call pairs:** keep the call and the result as separate canonical records if the result is absent.
- **Repeated snapshots:** deduplicate identical snapshots only when their fingerprints match exactly.
- **Multiple references to the same source object:** reuse the same canonical ID rather than duplicating evidence.
- **Missing timestamps entirely:** preserve the sequence and leave temporal fields null rather than synthesizing dates.

Deterministic behavior requirements

- No random UUID generation during normalization.

- No clock reads during normalization.
- No probabilistic ranking of candidate records.
- No semantic guessing from textual content.
- No external lookups.
- No hidden ordering dependence on hash map iteration.

12. Tradeoffs

The module makes a few deliberate tradeoffs in favor of trustworthiness and maintainability.

Tradeoff	Chosen direction	Why
Simplicity vs flexibility	Simplicity	A small fixed contract is easier to test and easier for downstream modules to trust.
Generity vs specificity	Specificity	The MVP only needs one generic JSON input format, so generic plugin architecture is wasteful.
Determinism vs heuristic inference	Determinism	Postmortems must be explainable; guessed structure is not evidence.
Lossless vs canonicalized	Controlled lossiness	Some source detail can be preserved as opaque evidence without polluting the canonical model.
Strictness vs resilience	Conservative resilience	Recover when evidence is still usable; fail only when canonical structure would be corrupted.

Why this design is preferred

A more flexible system would try to accommodate every odd trace shape through inheritance trees, rule engines, or adaptive inference. That would produce a large, fragile module whose behavior is difficult to reason about. The preferred design is boring in the best way: small, explicit, deterministic, and easy to audit.

13. Risks and Failure Modes

Normalization looks simple until real traces show up. Then the edge cases start multiplying.

Hidden complexity

- Temporal ordering may not match causal ordering.
- Source records may contain nested repeated objects that look identical but are not semantically equal.
- A single source field may carry multiple meanings depending on record kind.
- Parent-child trees may actually be DAGs or contain accidental cycles.
- Duplicate identifiers can appear in traces generated by agent retries or partial replays.

Ambiguous cases

Ambiguous case	Risk	Required behavior
Same ID, different payloads	Incorrect merge	Keep first-seen record and emit conflict diagnostics.
No parent, nested structure only	False inference	Preserve nesting in provenance, not as canonical parentage.
Missing time fields	Fake chronology	Leave timestamps null and rely on explicit order.
Unknown record type	Silent data loss	Preserve as opaque extension record when safe.
Conflicting timestamps	Hidden reordering	Use deterministic precedence and report the conflict.

Unsupported inputs

The module should reject traces that cannot be safely normalized into the canonical model, such as:

- non-iterable record collections;
- cycles in required acyclic relations that cannot be cut deterministically;
- records with irreconcilable identity collisions;
- malformed source metadata that prevents provenance tracking.

Maintenance risks

- Expanding the canonical model too early will make normalization logic harder to keep stable.
- Allowing downstream modules to depend on raw-source-specific behavior will undo the value of normalization.
- Adding heuristics “just for convenience” will create invisible coupling and make regression tests unreliable.

14. Testing Strategy

This module should be covered with fixture-first tests. The output must be easy to snapshot and diff.

Unit tests

Test area	What to verify
Record classification	Each supported source record kind maps to the correct canonical kind.
ID generation	The same source record always produces the same canonical ID.
Timestamp precedence	Explicit field precedence is stable and deterministic.
Reference resolution	Explicit links resolve correctly; missing links stay null.
Diagnostics emission	Conflicts, duplicates, and unknown kinds produce the expected diagnostics.

Integration tests

- Validate parser/validation output flowing into normalization.
- Confirm that the canonical package matches the canonical trace model contract.
- Check that downstream-friendly invariants hold across full trace fixtures.

Fixture-driven tests

Use fixtures that represent realistic and annoying traces:

Fixture type	Why it matters
Clean happy-path trace	Establishes the baseline mapping.
Duplicate records	Verifies deduplication and conflict handling.
Out-of-order events	Verifies ordering rules.
Missing timestamps	Verifies conservative fallback behavior.
Unknown record kinds	Verifies opaque preservation.
Cyclic reference case	Verifies graph-safety behavior.

Regression protection

- Snapshot the full normalized package for representative fixtures.
- Snapshot diagnostics separately so new warnings are deliberate and reviewable.
- Lock canonical schema version in tests so accidental schema drift fails fast.
- Add a small set of “nasty” traces that are expected to stay fixed forever.

15. Implementation Checklist

A practical implementation should follow this order:

1. Freeze the canonical schema version that this module targets.
2. Define the normalization input contract from Parser + Validation.
3. Define the normalization result wrapper and diagnostics schema.
4. Implement record classification for the MVP record kinds only.
5. Implement stable canonical ID derivation.
6. Implement timestamp precedence and ordering rules.
7. Implement explicit reference resolution.
8. Build graph assembly and cycle handling.
9. Preserve source provenance for every important field.
10. Emit structured diagnostics and summary counts.
11. Add fixture-based snapshot tests before widening record coverage.
12. Document every normalization rule that changes or drops information.
13. Verify the final canonical package is immutable from downstream code.

16. Future Expansion

The natural next steps should extend the same deterministic core, not replace it.

Future expansion	Why it follows naturally
Additional adapter inputs	The same normalization contract can absorb new source families later.
OpenTelemetry span normalization	Fits the same record-to-canonical mapping pattern once the MVP is stable.
Streaming normalization	Useful for very large traces, but only after the batch contract is proven.
Richer provenance views	Helpful for debugging and UI tooling without changing the canonical model.
Multi-run comparison support	Requires stable canonical identities, which this module already establishes.
Optional normalization profiles	Could support compatibility modes, but only after the base contract is frozen.

Anything that starts to infer meaning, score behavior, or compare runs is no longer just normalization. That work belongs downstream.

17. Final Recommendation

This design is MVP-ready if, and only if, the implementation stays conservative about inference and strict about provenance. The module should normalize what is explicit, preserve what is useful, and reject what would make the canonical model unreliable.

The main simplification still recommended is this: keep the MVP record taxonomy small. Do not add extra canonical kinds unless a real fixture forces the need. The correct shape for this module is a narrow, deterministic transform with strong diagnostics, not a general-purpose trace compiler.

In short: freeze the contract, keep the rules explicit, preserve evidence, and do not get clever.